

Universidad del Valle de Guatemala

CC2008 - Programación Orientada a Objetos

Semestre II, 2026

Temario A



Análisis de Laboratorio 1

Josue David Garcia Mendez 26375

1. Análisis

1.1 Requisitos funcionales

RF1. Gestión de entrenadores.

El sistema debe permitir realizar un enfrentamiento entre dos entrenadores, donde cada entrenador participe con cuatro Digimon distintos seleccionados al inicio de la batalla.

RF2. Características de los Digimon.

El sistema debe almacenar para cada Digimon su nombre, tipo, ataque, defensa y Digievolución. Los tipos disponibles son Fuego, Agua, Planta y Eléctrico.

RF3. Características de las Digievoluciones.

El sistema debe almacenar para cada Digievolución su nombre, efecto y probabilidad de activación. El efecto puede aumentar el ataque, aumentar la defensa o causar daño directamente al enemigo.

RF4. Desarrollo de la batalla.

El sistema debe desarrollar el enfrentamiento durante un total de cuatro rondas.

RF5. Selección de Digimon por ronda.

En cada ronda, cada entrenador debe seleccionar uno de sus Digimon que no haya utilizado en una ronda anterior, evitando que un mismo Digimon sea utilizado más de una vez durante la batalla.

RF6. Selección de acción.

En cada ronda, el sistema debe permitir decidir entre realizar un ataque o utilizar la habilidad especial correspondiente a la Digievolución del Digimon seleccionado.

RF7. Aplicación del efecto de tipo.

Al calcular el poder de un ataque, el sistema debe considerar la relación entre los tipos de los Digimon. Las ventajas establecidas son Fuego sobre Planta, Planta sobre Agua, Agua sobre Fuego y Eléctrico sobre Agua.

RF8. Modificación del ataque según el tipo.

El sistema debe sumar 20 puntos cuando el ataque sea efectivo, restar 10 puntos cuando sea débil y no modificar el valor cuando la relación entre los tipos sea neutral.

RF9. Activación probabilística de la Digievolución.

Cuando se utilice una Digievolución, el sistema debe determinar su activación mediante su probabilidad asociada, generando un valor entre 0 y 100 y comprobando si se encuentra dentro del rango de activación correspondiente.

RF10. Aplicación de efectos de Digievolución.

Cuando una Digievolución se active, el sistema debe aplicar el efecto que tenga asociado. Este puede modificar el ataque, modificar la defensa o reducir directamente el ataque total del rival.

RF11. Persistencia de la Digievolución.

Cuando una Digievolución se active, su efecto debe aplicarse durante la ronda actual y mantenerse para la ronda siguiente.

RF12. Determinación del ganador de una ronda.

Al finalizar cada ronda, el sistema debe comparar los valores de ataqueTotal de ambos participantes y declarar ganador al entrenador cuyo Digimon obtenga el valor más alto.

RF13. Empate de una ronda.

Si ambos Digimon obtienen el mismo valor de ataqueTotal, el sistema debe registrar la ronda como un empate.

RF14. Determinación del ganador de la batalla.

Después de finalizar las cuatro rondas, el sistema debe determinar como ganador de la batalla al entrenador que haya ganado la mayor cantidad de rondas.

1.2 Descripción de las clases

Digimon

Representa a cada Digimon que puede participar en una batalla. Su propósito es almacenar las características necesarias para el combate, incluyendo su nombre, tipo, ataque, defensa y Digievolución. También permite determinar el efecto producido por la relación entre su tipo y el tipo del Digimon rival. Estas características corresponden directamente a las propiedades requeridas para cada Digimon en el enunciado.

Digievolucion

Representa la habilidad especial asociada a un Digimon. Su propósito es almacenar la información necesaria para identificar la habilidad, el efecto que produce, la magnitud de dicho efecto y su probabilidad de activación. Además, permite determinar de forma aleatoria si la habilidad logra activarse cuando el entrenador decide utilizarla. El efecto de una Digievolución puede aumentar el ataque, aumentar la defensa o afectar directamente el ataque total del rival.

Entrenador

Representa a cada uno de los dos participantes de la batalla. Su propósito es administrar el equipo de cuatro Digimon perteneciente al entrenador, controlar cuáles de ellos ya han sido utilizados y registrar la cantidad de rondas ganadas. También conserva temporalmente los efectos de una Digievolución que deban mantenerse para la ronda siguiente. Esto permite cumplir con la regla de utilizar un Digimon diferente en cada ronda y mantener los efectos que persisten entre rondas.

Batalla

Representa el enfrentamiento completo entre los dos entrenadores. Su propósito es mantener a los participantes y controlar el avance de las cuatro rondas de la batalla. También permite determinar al ganador final mediante la comparación de la cantidad de rondas ganadas por cada entrenador.

ControladorBatalla

Se encarga de coordinar y ejecutar la lógica necesaria para desarrollar cada ronda de la batalla. Su propósito es validar que los Digimon seleccionados puedan utilizarse, obtener sus características, calcular los valores de ataque total, comprobar la activación de las Digievoluciones, aplicar sus efectos y registrar al ganador de cada ronda. También sirve como intermediario entre el programa principal y los objetos que representan la batalla.

Tipo

Es una enumeración que representa los tipos elementales permitidos para los Digimon: FUEGO, AGUA, PLANTA y ELECTRICO. Su propósito es limitar el tipo de un Digimon a los valores establecidos por el problema y facilitar la evaluación de ventajas, desventajas y relaciones neutrales entre tipos. Los cuatro tipos provienen directamente del enunciado.

EfectoDigievolucion

Es una enumeración que representa las distintas clases de efectos que puede producir una Digievolución: AUMENTAR_ATAQUE, AUMENTAR_DEFENSA y DANIO_DIRECTO. Su propósito es identificar de manera controlada qué comportamiento debe aplicar el sistema cuando una habilidad especial logra activarse. Estos tres tipos de efectos corresponden a los establecidos en el problema.

Main

Es la clase principal y el punto de entrada del programa. Su propósito es crear los objetos necesarios para iniciar la batalla, recibir las decisiones de los usuarios, mostrar la información de las rondas y presentar los resultados. Delega la lógica del enfrentamiento al controlador y a las demás clases del sistema, evitando concentrar en la clase principal las responsabilidades propias del modelo.

1.3 Descripción de atributos

1.3.1 Clase Digimon

Atributo	Tipo	Propósito
nombre	String	Almacena el nombre que identifica al Digimon.
tipo	Tipo	Almacena el tipo elemental utilizado para determinar ventajas y desventajas.
ataque	int	Almacena el valor de ataque base del Digimon.
defensa	int	Almacena el valor de defensa base del Digimon.
digievolucion	Digievolucion	Mantiene la Digievolución asociada al Digimon.

1.3.2 Clase Digievolucion

Atributo	Tipo	Propósito
nombre	String	Almacena el nombre que identifica la Digievolución, por ejemplo, Adult, Mega o Ultimate.
tipoEfecto	EfectoDigievolucion	Indica qué clase de efecto produce la Digievolución cuando logra activarse: aumentar ataque, aumentar defensa o causar daño directo al rival.
cantidadEfecto	int	Almacena la magnitud numérica del efecto producido por la Digievolución, permitiendo representar valores como +15 de ataque, +20 de defensa o 10 puntos de daño directo.
probabilidadActivacion	int	Almacena la probabilidad de que la Digievolución se active cuando se intenta utilizar. Se expresa mediante un valor asociado al rango de 0 a 100.

1.3.3 Clase Entrenador

Atributo	Tipo	Propósito
nombre	String	Almacena el nombre que identifica al entrenador durante la batalla.
digimones	Digimon[]	Almacena el equipo de cuatro Digimon pertenecientes al entrenador y permite acceder a ellos mediante su posición.
utilizados	boolean[]	Registra cuáles Digimon del equipo ya participaron en una ronda. Cada posición corresponde a un Digimon y permite impedir que sea seleccionado nuevamente.
rondasGanadas	int	Almacena la cantidad de rondas ganadas por el entrenador y permite determinar al ganador de la batalla después de las cuatro rondas.
bonoAtaquePendiente	int	Almacena temporalmente una modificación de ataque que debe aplicarse en la siguiente ronda debido al efecto de una Digievolución activada previamente.

defensaExtraPendiente	int	Almacena temporalmente un aumento de defensa producido por una Digievolución que debe mantenerse durante la siguiente ronda.
------------------------------	-----	--

1.3.4 Clase Batalla

Atributo	Tipo	Propósito
entrenador1	Entrenador	Mantiene una referencia al primer entrenador participante de la batalla.
entrenador2	Entrenador	Mantiene una referencia al segundo entrenador participante de la batalla.
rondaActual	int	Almacena el número de la ronda que se está desarrollando y permite controlar que el enfrentamiento finalice después de cuatro rondas.

1.3.5 Clase ControladorBatalla

Atributo	Tipo	Propósito
batalla	Batalla	Mantiene la instancia de la batalla que está siendo administrada por el controlador y permite acceder a su estado, rondas y entrenadores.
ultimoAtaqueTotal1	int	Almacena el ataqueTotal obtenido por el Digimon del primer entrenador en la última ronda válida.
ultimoAtaqueTotal2	int	Almacena el ataqueTotal obtenido por el Digimon del segundo entrenador en la última ronda válida.
habilidad1Activada	boolean	Indica si la habilidad especial seleccionada por el primer entrenador logró activarse durante la última ronda.
habilidad2Activada	boolean	Indica si la habilidad especial seleccionada por el segundo entrenador logró activarse durante la última ronda.
ultimaRondaValida	boolean	Indica si las selecciones realizadas permitieron ejecutar correctamente la última ronda, evitando procesar una ronda con Digimon inexistentes o previamente utilizados.

1.3.6 Enumeración Tipo

Valor	Propósito
FUEGO	Representa a los Digimon de tipo Fuego.
AGUA	Representa a los Digimon de tipo Agua.
PLANTA	Representa a los Digimon de tipo Planta.
ELECTRICO	Representa a los Digimon de tipo Eléctrico.

1.3.7 Enumeración EfectoDigievolucion

VALOR	PROPÓSITO
AUMENTAR_ATAQUE	Representa una Digievolución cuyo efecto incrementa el ataque.
AUMENTAR_DEFENSA	Representa una Digievolución cuyo efecto incrementa la defensa.
DANIO_DIRECTO	Representa una Digievolución cuyo efecto reduce directamente el ataqueTotal del rival.

1.4 Descripción de métodos

Modelos:

Digimon

Método	Parámetros	Retorno	Propósito
Digimon(...)	nombre, tipo, ataque, defensa, digievolucion	—	Construye un nuevo Digimon inicializando su nombre, tipo, ataque, defensa y la Digievolución asociada.
getNombre()	Ninguno	String	Devuelve el nombre del Digimon.

getTipo()	Ninguno	Tipo	Devuelve el tipo elemental del Digimon.
getAtaque()	Ninguno	int	Devuelve el ataque base del Digimon.
getDefensa()	Ninguno	int	Devuelve la defensa base del Digimon.
getDigievolucion()	Ninguno	Digievolucion	Devuelve la Digievolución asociada al Digimon.
calcularEfectoTipo(rival)	Digimon rival	int	Determina el modificador de tipo según la relación entre el tipo propio y el del rival, devolviendo +20, -10 o 0.

Digievolucion

Método	Parámetros	Retorno	Propósito
Digievolucion(...)	nombre, tipoEfecto, cantidadEfecto, probabilidadActivacion	—	Construye una nueva Digievolución inicializando su nombre, el tipo de efecto que produce, la magnitud de ese efecto y su probabilidad de activación.
seActiva()	Ninguno	boolean	Genera un número aleatorio entre 0 y 100 y determina si la habilidad se activa, comparándolo contra la probabilidad

			de activación almacenada.
getNombre()	Ninguno	String	Devuelve el nombre de la Digievolución .
getTipoEfecto()	Ninguno	EfectoDigievolucion	Devuelve el tipo de efecto que produce esta Digievolución al activarse.
getCantidadEfecto()	Ninguno	int	Devuelve la magnitud numérica del efecto (puntos de ataque, defensa o daño).
getProbabilidadActivacion()	Ninguno	int	Devuelve el porcentaje de probabilidad de que la habilidad se active.

Entrenador

Método	Parámetros	Retorno	Propósito
Entrenador(nombre)	String nombre	—	Construye un nuevo entrenador con el nombre indicado y sus 4 espacios de Digimon inicialmente vacíos.
agregarDigimon(digimon, posicion)	Digimon digimon, int posicion	void	Asigna un Digimon a una posición específica del equipo del entrenador,

validando que la posición exista.

getDigimon(posicion)	int posicion	Digimon	Devuelve el Digimon almacenado en la posición indicada del equipo.
fueUtilizado(posicion)	int posicion	boolean	Indica si el Digimon de esa posición ya fue seleccionado en una ronda anterior de la batalla.
marcarComoUtilizado(posicion)	int posicion	void	Marca la posición indicada como utilizada, para impedir que ese Digimon se vuelva a elegir.
ganarRonda()	Ninguno	void	Incrementa en uno la cantidad de rondas que ha ganado el entrenador.
agregarBonoAtaque(valor)	int valor	void	Acumula un valor (positivo o negativo) al bono de ataque pendiente, producto de una Digievolución activada.
consumirBonoAtaque()	Ninguno	int	Devuelve el bono de ataque acumulado hasta el momento y lo reinicia a 0, ya que solo debe aplicarse una vez, en la siguiente ronda.
agregarDefensaExtra(valor)	int valor	void	Acumula un valor a la defensa extra pendiente, producto de una Digievolución de tipo aumentar defensa.
consumirDefensaExtra()	Ninguno	int	Devuelve la defensa extra acumulada y la reinicia a 0, para que solo se aplique en la ronda siguiente.
getNombre()	Ninguno	String	Devuelve el nombre del entrenador.

getRondasGanadas()	Ninguno	int	Devuelve la cantidad de rondas ganadas hasta el momento.
getDigimones()	Ninguno	Digimon[]	Devuelve el arreglo completo con los 4 Digimon del entrenador.

Batalla

Método	Parámetros	Retorno	Propósito
Batalla(entrenador1, entrenador2)	Entrenador entrenador1, Entrenador entrenador2	—	Construye la batalla entre los dos entrenadores recibidos e inicializa la ronda actual en 1.
avanzarRonda()	Ninguno	void	Incrementa en uno el número de la ronda actual, para pasar a la siguiente.
batallaTerminada()	Ninguno	boolean	Indica si ya se jugaron las 4 rondas de la batalla (rondaActual > 4).
obtenerGanador()	Ninguno	Entrenador	Compara las rondas ganadas de ambos entrenadores y devuelve al que tenga más; devuelve null si están empatados.
getEntrenador1()	Ninguno	Entrenador	Devuelve la referencia al primer entrenador de la batalla.
getEntrenador2()	Ninguno	Entrenador	Devuelve la referencia al segundo entrenador de la batalla.
getRondaActual()	Ninguno	int	Devuelve el número de la ronda en la que se encuentra la batalla actualmente.

Controlador:

ControladorBatalla

Método	Parámetros	Retorno	Propósito
ControladorBatalla(entrenador1, entrenador2)	Entrenador entrenador1, Entrenador entrenador2	—	Construye el controlador y crea internamente la Batalla que administrará entre los dos entrenadores recibidos.
puedeUsarDigimon(entrenador, posicion)	Entrenador entrenador, int posicion	boolean	Valida que la posición esté en el rango 0-3, que exista un Digimon asignado ahí y que no haya sido utilizado previamente.
seleccionarDigimon(entrenador, posicion)	Entrenador entrenador, int posicion	Digimon	Método privado que, si la selección es válida, marca el Digimon como utilizado y lo devuelve para participar en la ronda.
jugarRonda(entrenador1, posicion1, usarHabilidad1, entrenador2, posicion2, usarHabilidad2)	Entrenador entrenador1, int posicion1, boolean usarHabilidad1, Entrenador entrenador2, int posicion2, boolean usarHabilidad2	void	Resuelve una ronda completa: valida las selecciones, calcula el ataqueTotal de cada Digimon (ataque base, efecto de tipo, defensa del rival y bonos pendientes), aplica la Digievolución de cada entrenador si decide usarla y se activa, y registra la ronda ganada al entrenador con mayor ataqueTotal.

avanzarRonda()	Ninguno	void	Le indica a la Batalla interna que avance a la siguiente ronda.
batallaTerminada()	Ninguno	boolean	Indica si ya se completaron las 4 rondas de la batalla.
obtenerGanadorFinal()	Ninguno	Entrenador	Devuelve el entrenador ganador de la batalla completa, obtenido desde la Batalla interna.
getRondaActual()	Ninguno	int	Devuelve el número de ronda en la que va la batalla.
isUltimaRondaValida()	Ninguno	boolean	Indica si las selecciones de Digimon de la última ronda jugada fueron válidas.
getUltimoAtaqueTotal1()	Ninguno	int	Devuelve el ataqueTotal calculado para el entrenador 1 en la última ronda jugada.
getUltimoAtaqueTotal2()	Ninguno	int	Devuelve el ataqueTotal calculado para el entrenador 2 en la última ronda jugada.
isHabilidad1Activada()	Ninguno	boolean	Indica si la Digievolución del entrenador 1 se activó en la última ronda.
isHabilidad2Activada()	Ninguno	boolean	Indica si la Digievolución del entrenador 2 se

			activó en la última ronda.
getBatalla()	Ninguno	Batalla	Devuelve la instancia de Batalla administrada por el controlador.

Principal:

Main

Método	Parámetros	Retorno	Propósito
main(args)	String[] args	void	Punto de entrada del programa. Crea las Digievoluciones, los Digimon, los dos entrenadores, y el ControladorBatalla; ejecuta el ciclo de las 4 rondas pidiendo al usuario sus decisiones, y muestra los resultados de cada ronda y el resultado final de la batalla.
leerEnteroEnRango(scanner, mensaje, minimo, maximo)	Scanner scanner, String mensaje, int minimo, int maximo	int	Solicita un número entero al usuario de forma repetida hasta que ingrese un valor válido dentro del rango indicado, evitando errores por texto no numérico o fuera de rango.
mostrarDigimones(entrenador)	Entrenador entrenador	void	Imprime en pantalla la lista de los 4 Digimon del entrenador, indicando cuáles ya fueron utilizados.

seleccionarDigimonValido(scanner, controlador, entrenador)

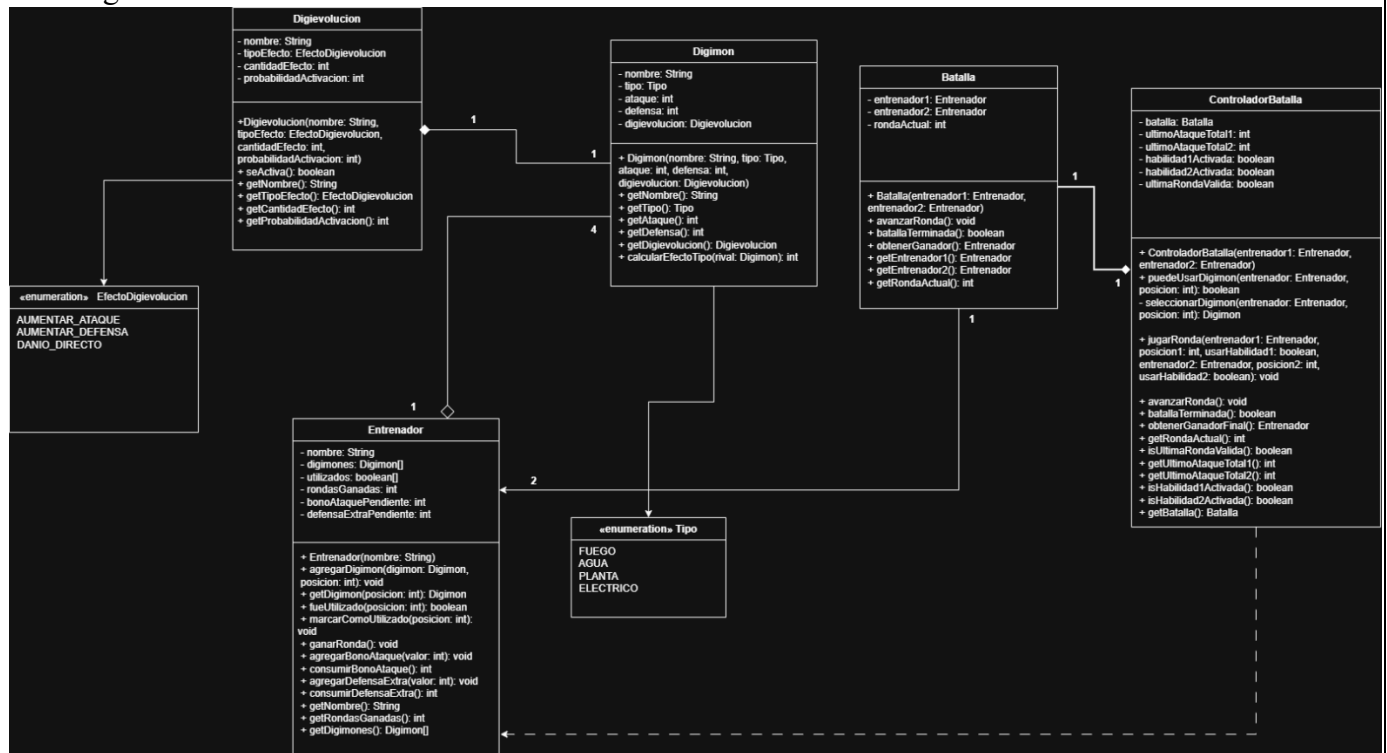
Scanner scanner,
ControladorBatalla
controlador,
Entrenador
entrenador

int

Solicita al usuario
que elija un Digimon
y repite la solicitud
hasta que la posición
elegida sea válida
(existente y no
utilizada
previamente),
devolviendo la
opción elegida (1 a
4).

2. Diseño

2.1 Diagrama de clases UML



2.2 Descripción de las relaciones

Clases relacionadas	Tipo de relación	Descripción
ControladorBatalla – Batalla	Composición	ControladorBatalla crea y mantiene la instancia de Batalla que administra. El controlador actúa como el objeto contenedor de la batalla durante la ejecución.
Batalla – Entrenador	Asociación	Una batalla mantiene referencias a los dos entrenadores participantes. Los entrenadores pueden existir independientemente de la batalla.
Entrenador – Digimon	Agregación	Un entrenador agrupa cuatro Digimon para participar en el enfrentamiento. Los Digimon pueden existir independientemente del entrenador.
Digimon – Digievolucion	Composición	Cada Digimon crea su propia Digievolución en el constructor y nadie más la comparte; la Digievolución no tiene un propósito fuera del Digimon que la posee.
Digimon – Tipo	Asociación	Digimon utiliza la enumeración Tipo para representar su elemento y determinar las relaciones de ventaja, desventaja o neutralidad.
Digievolucion – EfectoDigievolucion	Asociación	Digievolucion utiliza esta enumeración para identificar el tipo de efecto que debe aplicar cuando la habilidad se activa.

3. Consideraciones de implementación

3.1 Cálculo del ataque total

El ataque total de cada Digimon se calcula tomando como base su valor de ataque y aplicando el modificador correspondiente a la relación de tipos. Si el ataque es efectivo, se agregan 20 puntos; si es débil, se restan 10 puntos; y si es neutral, no se aplica ninguna modificación. Además, se considera la defensa del Digimon rival y los efectos activos o pendientes producidos por las Digievoluciones.

Debido a que el enunciado establece que cada Digimon posee defensa, pero no especifica una fórmula para incorporarla al combate, se decidió utilizar la defensa del rival como una reducción directa sobre el ataque total. De esta manera, tanto la defensa base como los efectos de Digievolución que la aumentan influyen en el resultado de la ronda.

La fórmula utilizada para realizar el cálculo es la siguiente:

$\text{ataqueTotal} = \text{ataqueBase} + \text{efectoTipo} - \text{defensaRival} + \text{efectosDigievolucion}$

3.2 Manejo de Digievoluciones

Cada Digimon posee una Digievolución con un efecto y una probabilidad de activación. Cuando un entrenador decide utilizar la habilidad especial, el sistema genera un número aleatorio entre 0 y 100 y determina si la habilidad se activa de acuerdo con su probabilidad. Si la Digievolución no se activa, no se aplica su efecto. Si se activa, el efecto correspondiente se aplica durante la ronda actual y se conserva para aplicarse nuevamente en la ronda siguiente. Después de aplicarse en la siguiente ronda, el efecto pendiente es eliminado.

3.3 Control de Digimon utilizados

Cada entrenador dispone de cuatro Digimon y cada uno puede utilizarse solamente una vez durante la batalla. Para controlar esta restricción, el sistema mantiene un registro de los Digimon que ya participaron. Antes de aceptar una selección se comprueba que el Digimon exista y que no haya sido utilizado previamente. Una vez seleccionado para una ronda válida, se marca como utilizado y no puede volver a seleccionarse.

3.4 Control de las rondas y ganador

La batalla comienza en la ronda 1 y continúa hasta completar cuatro rondas. Una ronda únicamente avanza después de haberse ejecutado correctamente. Al finalizar cada ronda, el entrenador cuyo Digimon obtenga el mayor ataque total suma una ronda ganada; si ambos valores son iguales, ninguno obtiene una victoria. Después de completar las cuatro rondas se comparan las rondas ganadas por ambos entrenadores para determinar al ganador de la batalla. Si ambos obtienen la misma cantidad de victorias, la batalla finaliza en empate.